LoongSocket LS-W1 S3 N16R8 API Reference

README.md

LoongSocket LS-W1 S3 N16R8 API

This folder documents the firmware API used by the

FW-LS-W1-S3-N16R8 project.

Module

• Project: FW-LS-W1-S3-N16R8

• Module P/N: LS-W1-1119-38P-S3U-N16R8

• MCU: ESP32-S3

• Flash: 16 MB

• PSRAM: 8 MB

• ADC: MCP3208, 8 channels, 12 bit, 2500 mV reference

• IDF: 5.5.4

Components

Component	Header	Purpose
BMS	ls_bms.h	Board identity, reset reason, memory and safe-mode status
USB	ls_usb.h	Native USB Serial/JTAG status and recovery
Storage	ls_storage.h	NVS-backed storage with write-on-change
MCP3208	ls_mcp3208.h	External ADC acquisition, filtering, calibration and diagnostics
I/O	ls_io.h	GPIO, PWM and board reserved-pin protection
Board Test	ls_board_test.h	Production/demo helpers for PWM, square-wave and ADC PASS/NO PASS

Startup order

Recommended initialization order:

```
ls_bms_init();
ls_usb_init();
ls_mcp3208_init();
ls_io_init();
ls_storage_init();
```

The current Kickstarter monitor demo follows this order in main/main.cpp.

Documentation Files

• LS_BMS_API.md

• LS_USB_API.md

• LS_STORAGE_API.md

• LS_MCP3208_API.md

• LS_IO_API.md

• LS_BOARD_TEST_API.md

• LS_API_STATUS.md

LS_API_STATUS.md

LS API Status

Status date: 2026-08-29

Implemented

Area	Status
BMS identity/status	Implemented
USB Serial/JTAG status	Implemented
USB recovery hook	Implemented, basic
NVS storage wrappers	Implemented
Write-on-change storage	Implemented
MCP3208 single read	Implemented
MCP3208 all-channel read	Implemented
MCP3208 burst read	Implemented
MCP3208 filter/calibration/range hooks	Implemented
MCP3208 hardware info	Implemented
MCP3208 diagnostics	Implemented
GPIO configure/read/write	Implemented
Reserved-pin protection	Implemented
Reserved-pin listing	Implemented
PWM start/set/stop	Implemented
PWM status query	Implemented
Board routing test helpers	Implemented
Kickstarter monitor demo	Implemented

Pending / Future

Area	Notes
USB command console	Needed for production/service workflow
USB locked service commands	Needed before exposing dangerous operations
Boot/recovery procedure	Needs final policy for products without EN/BOOT access
Production report format	CSV/PDF or JSON report still pending
License/provisioning integration	Future factory workflow
Network/WiFi API	Not part of this first S3 API slice
Ethernet SPI option	Deferred

Current Validation

• Build validated with ESP-IDF 5.5.4.

• Monitor demo flashed and checked on COM8.

• S3 module detected as ESP32-S3 with 16 MB flash and 8 MB PSRAM.

• MCP3208 routing previously tested on CH0..CH7.

LS_BMS_API.md

LS BMS API

Header: components/ls_bms/include/ls_bms.h

The BMS component gives basic board-management information. It is the first API to initialize because other services can report their status through it later.

Main Types

ls_bms_status_t

• chip_model

• mac

• reset_reason

• free_heap_bytes

• minimum_free_heap_bytes

• psram_bytes

• boot_failure_count

• nvs_ready

• safe_mode_recommended

• usb_ready

• usb_connected

• usb_recovery_available

Functions

```
esp_err_t ls_bms_init(void);
esp_err_t ls_bms_get_status(ls_bms_status_t *status);
esp_err_t ls_bms_mark_healthy(void);
esp_err_t ls_bms_clear_safe_mode(void);
const char *ls_bms_reset_reason_name(esp_reset_reason_t reason);
```

Notes

• ls_bms_mark_healthy() should be called after the application has completed a

stable startup.

• safe_mode_recommended is intended for future protection/recovery logic.

• Current demo logs chip, MAC, reset reason, heap and PSRAM.

LS_USB_API.md

LS USB API

Header: components/ls_usb/include/ls_usb.h

The USB component tracks the native ESP32-S3 USB Serial/JTAG service and exposes

a small recovery API.

Main Types

ls_usb_status_t

• initialized

• driver_owned

• connected

• init_failures

• recovery_attempts

• last_error

Functions

```
esp_err_t ls_usb_init(void);
esp_err_t ls_usb_get_status(ls_usb_status_t *status);
esp_err_t ls_usb_recover(void);
```

Notes

• The S3 module has no easy external access to EN/BOOT in the final product, so USB robustness is important.

• Keep USB commands explicit and safe. Future production commands should be gated by service mode or password.

LS_STORAGE_API.md

LS Storage API

Header: components/ls_storage/include/ls_storage.h

The storage component wraps NVS and only commits values when their content changes. This reduces unnecessary flash writes.

Main Types

ls_storage_status_t

• initialized

• committed_writes

• skipped_writes

• last_error

Functions

```
esp_err_t ls_storage_init(void);
esp_err_t ls_storage_get_status(ls_storage_status_t *status);

esp_err_t ls_storage_get_blob(const char *name_space, const char *key,
                              void *value, size_t *length);
esp_err_t ls_storage_set_blob(const char *name_space, const char *key,
                              const void *value, size_t length);

esp_err_t ls_storage_get_string(const char *name_space, const char *key,
                                char *value, size_t *length);
esp_err_t ls_storage_set_string(const char *name_space, const char *key,
                                const char *value);

esp_err_t ls_storage_get_u32(const char *name_space, const char *key,
                             uint32_t *value);
esp_err_t ls_storage_set_u32(const char *name_space, const char *key,
                             uint32_t value);

esp_err_t ls_storage_erase_key(const char *name_space, const char *key);
esp_err_t ls_storage_erase_namespace(const char *name_space);
```

Notes

• Use short namespaces and keys.

• Use ls_storage_get_status() during endurance tests to see committed and skipped writes.

LS_MCP3208_API.md

LS MCP3208 API

Header: components/ls_mcp3208/include/ls_mcp3208.h

This is the preferred public name for the external ADC API. Older ls_adc.h symbols remain available for compatibility.

Hardware Profile

• ADC: MCP3208

• Channels: 8

• Resolution: 12 bit

• Reference: 2500 mV

• SPI host: SPI2

• CS: GPIO3

• MOSI: GPIO11

• CLK: GPIO12

• MISO: GPIO13

Main Types

The ls_mcp3208_* types are aliases of the compatibility ls_adc_* types:

• ls_mcp3208_filter_t

• ls_mcp3208_filter_config_t

• ls_mcp3208_calibration_t

• ls_mcp3208_range_t

• ls_mcp3208_sample_t

• ls_mcp3208_status_t

• ls_mcp3208_burst_t

• ls_mcp3208_hardware_info_t

Main Functions

```
ls_mcp3208_init();
ls_mcp3208_get_hardware_info(info);
ls_mcp3208_get_status(status);
ls_mcp3208_reset_diagnostics();

ls_mcp3208_read_raw(channel, raw);
ls_mcp3208_sample(channel, sample);
ls_mcp3208_read_raw_all(raw);
ls_mcp3208_sample_all(samples);

ls_mcp3208_read_burst(channel, sample_count, burst);
ls_mcp3208_read_burst_all(sample_count, bursts);

ls_mcp3208_set_filter(channel, config);
ls_mcp3208_get_filter(channel, config);
ls_mcp3208_reset_filter(channel);

ls_mcp3208_set_calibration(channel, calibration);
ls_mcp3208_get_calibration(channel, calibration);
ls_mcp3208_set_range(channel, range);
ls_mcp3208_get_range(channel, range);
```

Limits

• Channels: 0..7

• Burst samples: max LS_MCP3208_MAX_BURST_SAMPLES

• Raw counts: 0..4095

Notes

• ls_mcp3208_read_burst() is used by the board-test component, so production validation and application code share the same acquisition path.

• Direct PWM-to-ADC wiring validates routing, not precision calibration.

LS_IO_API.md

LS I/O API

Header: components/ls_io/include/ls_io.h

The I/O API controls GPIO and PWM while protecting board-reserved pins.

Reserved Pins

The current board profile reserves:

GPIO	Reason
0	boot strap
3	MCP3208 chip select
11	MCP3208 MOSI
12	MCP3208 clock
13	MCP3208 MISO
19	USB D-
20	USB D+

Main Types

- ls_io_mode_t
- ls_io_pull_t
- ls_io_config_t
- ls_io_status_t
- ls_io_reserved_pin_t
- ls_io_pwm_status_t

Functions

```
esp_err_t ls_io_init(void);
esp_err_t ls_io_get_status(ls_io_status_t *status);

esp_err_t ls_io_get_reservation(int pin, const char **reservation);
const ls_io_reserved_pin_t *ls_io_reserved_pins(uint8_t *count);

esp_err_t ls_io_configure(const ls_io_config_t *config);
esp_err_t ls_io_write(int pin, bool level);
esp_err_t ls_io_read(int pin, bool *level);

esp_err_t ls_io_pwm_start(int pin, uint32_t frequency_hz, uint16_t duty);
esp_err_t ls_io_pwm_set_duty(int pin, uint16_t duty);
esp_err_t ls_io_pwm_stop(int pin);
esp_err_t ls_io_pwm_get_status(int pin, ls_io_pwm_status_t *status);
uint8_t ls_io_pwm_get_active(ls_io_pwm_status_t *status, uint8_t max_entries);
```

PWM Limits

- PWM channels: LS_IO_MAX_PWM_CHANNELS
- Frequency: 10..20000 Hz
- Duty: 0..10000

Notes

- PWM claims a pin until ls_io_pwm_stop() is called.
- ls_io_get_reservation() returns ESP_ERR_NOT_SUPPORTED for protected pins.

LS_BOARD_TEST_API.md

LS Board Test API

Header: components/ls_board_test/include/ls_board_test.h

The board-test API is a helper layer for production checks, laboratory routing tests and the Kickstarter monitor demo.

PWM Sources

The helper starts two PWM sources:

Signal	GPIO	Frequency	Duty
PWM_CH1	GPIO6	8 kHz	38.00 %
PWM_CH2	GPIO7	8 kHz	76.00 %

Square-Wave Pins

The helper can output a 2 Hz square wave on user GPIOs. Reserved ADC SPI, native USB and BOOT pins are excluded.

Main Types

- ls_board_test_adc_config_t
- ls_board_test_adc_channel_t
- ls_board_test_adc_report_t
- ls_board_test_pin_t

Functions

```
const ls_board_test_adc_config_t *ls_board_test_default_adc_config(void);
const ls_board_test_pin_t *ls_board_test_square_wave_pins(uint8_t *count);
esp_err_t ls_board_test_start_pwm_sources(void);
esp_err_t ls_board_test_start_square_wave_outputs(void);
esp_err_t ls_board_test_set_square_wave_outputs(bool level);
esp_err_t ls_board_test_read_adc_report(const ls_board_test_adc_config_t *config,
                                        ls_board_test_adc_report_t *report);
void ls_board_test_log_adc_report(const char *label,
                                  const ls_board_test_adc_config_t *config,
                                  const ls_board_test_adc_report_t *report);
```

PASS/NO PASS Meaning

- PASS | ready: no ADC input is connected, and no noise/saturation is detected.
- PASS | one channel detected: one ADC channel detects the PWM test signal.
- NO PASS: several channels are active/noisy/saturated or the ADC read failed.

This behavior is useful for demos because the board starts in a clean PASS state even before a test wire is connected.